As we can see at line #1 of listing 14.13, this generic definition of A* takes two extra arguments, a distance function, and a heuristic: they both contribute to the computation of the so-called *f-score*, at line #13: this value is a mix of the cost of reaching current node `u` from the source, and the expected cost needed in order to reach the goal from `u`.

By controlling these two arguments, we can obtain either BFS or Dijkstra (or neither); for both of them, the heuristic will need to be a function that is identically equal to `0`, something we could write like `lambda(v) → 0`; both algorithms, in fact, completely disregard any notion of or information about the distance of vertices to goal.

For what concerns the distance metrics, the situation is different:

- Dijkstra's algorithm uses the edge's weight as distance function, so we need to pass something like `distance = lambda(e) → e.weight`.
- BFS, instead, only takes into account the number of edges traversed, which is equivalent to considering all edges to have the same weight, identically equal to `1`! And thus, we can pass `distance = lambda(e) → 1`.

In practice, 99.9% of the times, you should better implement directly Dijkstra, or even more BFS, and not as a special case of A*: this brings us to a golden rule I learnt from a great engineer I worked with, about keeping things simple.